Practical 10

Thread Pools, wait/notify & Deadlock

Part A — Practice Programs (Hour 1)

1) Thread-Pool Runner — ExecutorService with 3 threads

Code

```
import java.util.concurrent.*;

public class PoolRunner {
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        for (int id = 1; id <= 10; id++) {
            final int taskId = id;
            pool.execute(() -> {
                System.out.println("Task " + taskId + " running on " +
                    Thread.currentThread().getName());
                try { Thread.sleep(100); } catch (InterruptedException e) { }
            });
        }
        pool.shutdown();
        boolean finished = pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("All tasks finished: " + finished + " (only 3 pool threads were used for
            10 tasks)");
    }
}
```

Output

```
Task 3 running on pool-1-thread-3
Task 1 running on pool-1-thread-1
Task 2 running on pool-1-thread-2
Task 4 running on pool-1-thread-1
Task 5 running on pool-1-thread-3
Task 6 running on pool-1-thread-2
Task 7 running on pool-1-thread-3
Task 8 running on pool-1-thread-1
Task 9 running on pool-1-thread-2
Task 10 running on pool-1-thread-1
All tasks finished: true (only 3 pool threads were used for 10 tasks)
```

2) Producer–Consumer — wait()/notify() on a small shared buffer

Code

```
public class ProducerConsumer {
    static class Buffer {
        private final int[] items = new int[3];
        private int count = 0, in = 0, out = 0;

        synchronized void put(int value) throws InterruptedException {
            while (count == items.length) wait();      // buffer full -> wait
            items[in] = value;
            in = (in + 1) % items.length;
            count++;
            System.out.println("Produced " + value + "   (buffer size " + count + ")");
            notifyAll();
        }

        synchronized int take() throws InterruptedException {
            while (count == 0) wait();                 // buffer empty -> wait
            int value = items[out];
            out = (out + 1) % items.length;
            count--;
            System.out.println("   Consumed " + value + " (buffer size " + count + ")");
            notifyAll();
            return value;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Buffer buffer = new Buffer();
        final int N = 8;
        Thread producer = new Thread(() -> {
            try { for (int i = 1; i <= N; i++) buffer.put(i); } catch (InterruptedException e) { }
        }, "producer");
        Thread consumer = new Thread(() -> {
            int sum = 0;
            try { for (int i = 1; i <= N; i++) sum += buffer.take(); } catch (InterruptedException
                e) { }
            System.out.println("Consumed all " + N + " items, sum = " + sum + " (expected " + (N *
```

```
                (N + 1) / 2) + ")");
        }, "consumer");
        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
    }
}
```

Output

```
Produced 1   (buffer size 1)
Produced 2   (buffer size 2)
Produced 3   (buffer size 3)
   Consumed 1 (buffer size 2)
   Consumed 2 (buffer size 1)
   Consumed 3 (buffer size 0)
Produced 4   (buffer size 1)
Produced 5   (buffer size 2)
Produced 6   (buffer size 3)
   Consumed 4 (buffer size 2)
   Consumed 5 (buffer size 1)
   Consumed 6 (buffer size 0)
Produced 7   (buffer size 1)
Produced 8   (buffer size 2)
   Consumed 7 (buffer size 1)
   Consumed 8 (buffer size 0)
Consumed all 8 items, sum = 36 (expected 36)
```

3) Deadlock (For Advanced Learners — home) — opposite lock order, then consistent order

Code

```
public class DeadlockDemo {
    static void pause() { try { Thread.sleep(200); } catch (InterruptedException e) { } }

    static Thread worker(String name, Object first, Object second) {
        Thread t = new Thread(() -> {
            synchronized (first) {
                System.out.println(name + " locked first resource");
                pause();
                synchronized (second) {
                    System.out.println(name + " locked second resource -> done");
                }
            }
        }, name);
        t.setDaemon(true);
        return t;
    }

    static void run(String title, boolean sameOrder) throws InterruptedException {
        Object printer = new Object();      // fresh locks for every run
        Object scanner = new Object();
        System.out.println("=== " + title + " ===");
        Thread t1 = worker("T1", printer, scanner);
        Thread t2 = sameOrder ? worker("T2", printer, scanner) : worker("T2", scanner, printer);
        t1.start();
        t2.start();
        t1.join(1500);
        t2.join(1500);
        if (t1.isAlive() || t2.isAlive()) {
            System.out.println("DEADLOCK: T1 is " + t1.getState() + ", T2 is " + t2.getState());
        } else {
            System.out.println("Both threads finished normally");
        }
    }

    public static void main(String[] args) throws InterruptedException {
        run("Opposite lock order", false);
        run("Consistent lock order (printer then scanner)", true);
    }
}
```

Output

```
=== Opposite lock order ===
T1 locked first resource
T2 locked first resource
DEADLOCK: T1 is BLOCKED, T2 is BLOCKED
=== Consistent lock order (printer then scanner) ===
T1 locked first resource
T1 locked second resource -> done
T2 locked first resource
T2 locked second resource -> done
Both threads finished normally
```

Scenario chosen: two workers that each need a printer and a scanner. The threads are daemon threads so the program can still end after the deadlock is reported.

Part B — Project Work (Hour 2): MiniBank

MiniBank gets a transaction engine: a fixed thread pool of 4 threads, a producer-consumer buffer using wait/notify, and a deadlock demonstration with its fix (always lock the account with the smaller accountNumber first).

1) TransactionProcessor — newFixedThreadPool(4), submit(), stop()

Code

service/TransactionProcessor.java

```
package service;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class TransactionProcessor {
    private final ExecutorService pool = Executors.newFixedThreadPool(4);

    public void submit(Runnable task) {
        pool.execute(task);
    }

    public void stop() throws InterruptedException {
        pool.shutdown();
        pool.awaitTermination(30, TimeUnit.SECONDS);
    }
}
```

3) TransactionBuffer — shared array buffer with wait() and notifyAll()

Code

service/TransactionBuffer.java

```
package service;

public class TransactionBuffer {
    public record Txn(String type, long amount) { }

    private final Txn[] items = new Txn[3];
    private int count = 0, in = 0, out = 0;

    public synchronized void put(Txn t) throws InterruptedException {
        while (count == items.length) {
            wait();                 // buffer full: producer waits
        }
        items[in] = t;
        in = (in + 1) % items.length;
        count++;
        notifyAll();                // wake the consumer
    }

    public synchronized Txn take() throws InterruptedException {
        while (count == 0) {
            wait();                 // buffer empty: consumer waits
        }
        Txn t = items[out];
        out = (out + 1) % items.length;
        count--;
        notifyAll();                // wake the producer
        return t;
    }
}
```

4) Transfers — unsafe order versus consistent lock order

Code

service/Transfers.java

```
package service;

import exception.BankException;
import model.Account;

public class Transfers {
    private static void pause() {
        try { Thread.sleep(200); } catch (InterruptedException e) { }
    }

    private static void move(Account from, Account to, long amount) {
        try {
            from.withdraw(amount);
            to.deposit(amount);
        } catch (BankException e) {
            System.out.println("Transfer error: " + e.getMessage());
        }
    }

    /** Locks 'from' first and 'to' second - opposite directions can deadlock. */
    public static void transferUnsafe(Account from, Account to, long amount) {
        synchronized (from) {
            pause();
            synchronized (to) {
                move(from, to, amount);
            }
        }
    }

    /** Always locks the account with the smaller accountNumber first - no deadlock. */
    public static void transferSafe(Account from, Account to, long amount) {
        Account first  = from.getAccountNumber().compareTo(to.getAccountNumber()) < 0 ? from : to;
        Account second = (first == from) ? to : from;
        synchronized (first) {
            pause();
            synchronized (second) {
                move(from, to, amount);
            }
        }
    }
}
```

2-4) main — pool, producer-consumer, deadlock created and fixed

Code

service/MiniBank.java

```
package service;

import exception.BankException;
import model.Account;

public class MiniBank {
    static void deadlockRun(String title, boolean safe) throws InterruptedException {
        Account a = new Account("ACC-A", 5000, 0.04);
        Account b = new Account("ACC-B", 5000, 0.04);
        System.out.println("=== " + title + " ===");
        Thread t1 = new Thread(() -> {
            if (safe) Transfers.transferSafe(a, b, 100); else Transfers.transferUnsafe(a, b, 100);
        }, "A-to-B");
        Thread t2 = new Thread(() -> {
            if (safe) Transfers.transferSafe(b, a, 200); else Transfers.transferUnsafe(b, a, 200);
        }, "B-to-A");
        t1.setDaemon(true);
        t2.setDaemon(true);
        t1.start();
        t2.start();
        t1.join(1500);
        t2.join(1500);
        if (t1.isAlive() || t2.isAlive()) {
            System.out.println("DEADLOCK: " + t1.getName() + " is " + t1.getState() + ", " +
                t2.getName() + " is " + t2.getState());
        } else {
            System.out.println("Both transfers finished. ACC-A=" + a.getBalance() + ", ACC-B=" +
                b.getBalance()
                    + " (expected 5100 and 4900)");
        }
    }

    public static void main(String[] args) throws Exception {
        // 1-2. Thread pool processing a batch of deposits and withdrawals
        System.out.println("=== Thread pool: 20 transactions on 4 threads ===");
        Account acc = new Account("ACC-1", 1000, 0.04);
```

```
        TransactionProcessor processor = new TransactionProcessor();
        for (int i = 1; i <= 20; i++) {
            final int id = i;
            processor.submit(() -> {
                try {
                    if (id % 2 == 0) acc.withdraw(10); else acc.deposit(25);
                } catch (BankException e) {
                    System.out.println("Task " + id + " failed: " + e.getMessage());
                }
            });
        }
        processor.stop();
        System.out.println("Final balance: " + acc.getBalance() + " (expected 1000 + 10*25 - 10*10
            = 1150)");

        // 3. Producer-consumer with wait/notify
        System.out.println("=== Producer-consumer with wait()/notify() ===");
        TransactionBuffer buffer = new TransactionBuffer();
        Account target = new Account("ACC-2", 0, 0.04);
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 6; i++) {
                    buffer.put(new TransactionBuffer.Txn("DEPOSIT", i * 100));
                    System.out.println("Produced DEPOSIT " + (i * 100));
                }
            } catch (InterruptedException e) { }
        }, "producer");
        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 6; i++) {
                    TransactionBuffer.Txn t = buffer.take();
                    target.deposit(t.amount());
                    System.out.println("   Processed " + t.type() + " " + t.amount() + " -> balance
                        " + target.getBalance());
                }
            } catch (InterruptedException | BankException e) { }
        }, "consumer");
        producer.start();
        consumer.start();
        producer.join();
        consumer.join();

        // 4. Deadlock and its fix
        deadlockRun("Deadlock: opposite lock order", false);
        deadlockRun("Fixed: lock smaller accountNumber first", true);
    }
}
```

Output

```
=== Thread pool: 20 transactions on 4 threads ===
Final balance: 1150 (expected 1000 + 10*25 - 10*10 = 1150)
=== Producer-consumer with wait()/notify() ===
Produced DEPOSIT 100
Produced DEPOSIT 200
Produced DEPOSIT 300
Produced DEPOSIT 400
   Processed DEPOSIT 100 -> balance 100
Produced DEPOSIT 500
   Processed DEPOSIT 200 -> balance 300
Produced DEPOSIT 600
   Processed DEPOSIT 300 -> balance 600
   Processed DEPOSIT 400 -> balance 1000
   Processed DEPOSIT 500 -> balance 1500
   Processed DEPOSIT 600 -> balance 2100
=== Deadlock: opposite lock order ===
DEADLOCK: A-to-B is BLOCKED, B-to-A is BLOCKED
=== Fixed: lock smaller accountNumber first ===
Both transfers finished. ACC-A=5100, ACC-B=4900 (expected 5100 and 4900)
```

Package layout, build and tag

Code

```
minibank/
 |-- exception/
 |   |-- AccountNotFoundException.java
 |   |-- BankException.java
 |   |-- DailyLimitExceededException.java
 |   |-- InsufficientFundsException.java
 |   +-- InvalidAmountException.java
 |-- model/
 |   |-- annotation/
 |   |   |-- Id.java
 |   |   |-- MaxLength.java
 |   |   +-- Positive.java
 |   |-- Account.java
 |   |-- InterestBearing.java
 |   |-- Premium.java
 |   |-- PremiumAccount.java
 |   +-- Transactable.java
 |-- service/
 |   |-- AccountWorker.java
 |   |-- MiniBank.java
 |   |-- TransactionBuffer.java
 |   |-- TransactionProcessor.java
 |   |-- Transfers.java
 |   +-- WithdrawRule.java
 +-- util/
     |-- AnnotationValidator.java
     +-- Validator.java

javac -d out $(find exception model util service -name "*.java")
java -cp out service.MiniBank
cd out
jar --create --file minibank.jar --main-class=service.MiniBank .
java -jar minibank.jar
cd ..
git add .
git commit -m "Practical 10 - thread pool, wait/notify and deadlock"
git tag lab-10
```

The opposite-order version is detected as a deadlock (both threads BLOCKED); the fixed version finishes and both balances are correct (ACC-A=5100, ACC-B=4900).

Key Questions / Analysis / Interpretation

1. Why is a thread pool better than creating a new thread per task?

Creating and destroying a thread is expensive and an unlimited number of threads can exhaust memory. A pool starts a fixed number of worker threads once and reuses them for many tasks (the output shows only pool-1-thread-1 to 4 running 20 tasks), and queues extra tasks until a worker is free.

2. How do wait() and notify() coordinate two threads?

Inside a synchronized block a thread calls wait() to release the lock and sleep until a condition may have changed; another thread changes the state and calls notify()/notifyAll() to wake it. The producer waits when the buffer is full, the consumer waits when it is empty, and each wakes the other. The condition is always re-checked in a while loop.

3. What causes a deadlock, and how does consistent lock ordering prevent it?

A deadlock happens when each thread holds a lock the other needs and neither will release it (A holds ACC-A and waits for ACC-B while B holds ACC-B and waits for ACC-A). If every thread always acquires locks in the same global order (smaller accountNumber first), a circular wait cannot form, so one thread always gets both locks and finishes.